

Validity Challenges in Inferring AI Tool Behavior from Observational Pull Request Datasets

Ahmed Mursal
School of Computing
Edinburgh Napier University, UK
40646515@live.napier.ac.uk

Abstract—Observational datasets of AI-assisted pull requests face methodological challenges that can invalidate behavioral inferences. We analyze validity threats in the AIDev dataset (932,791 PRs) and demonstrate how agent attribution uncertainty, statistical clustering violations, measurement bias, and confounding limit interpretability. Through empirical examples, we show how apparent behavioral differences (98.5% vs. 19% test rates) may reflect artifacts rather than capabilities. We provide a methodological framework with requirements for reliable inference from observational AI tool data.

I. INTRODUCTION

Large-scale datasets of AI-assisted pull requests offer insights into real-world tool usage but face methodological challenges that can invalidate behavioral inferences. This study identifies and demonstrates validity threats in observational AI tool datasets, using AIDev (932,791 PRs) as a case study.

We focus on four critical threats: (1) agent attribution reliability, (2) statistical clustering violations, (3) measurement bias, and (4) confounding effects. Through empirical examples, we show how apparent behavioral differences may reflect artifacts rather than capabilities.

Contribution: A methodological framework identifying validity requirements for reliable inference from observational AI tool datasets.

II. VALIDITY THREATS IN OBSERVATIONAL AI TOOL RESEARCH

Observational AI tool datasets face four primary validity threats:

Attribution Reliability: Metadata and self-reports may misclassify tool usage due to incomplete records, mixed usage, or detection errors.

Statistical Clustering: Users, repositories, and organizations create correlated observations that violate independence assumptions, inflating significance tests.

Measurement Bias: Keyword-based outcome detection produces systematic errors, particularly across different languages and practices.

Confounding Effects: Developers select tools based on projects, experience, and context, making tool effects impossible to isolate without controls.

III. EMPIRICAL DEMONSTRATIONS

A. Attribution Evidence

Table ?? shows extreme user count imbalances suggesting attribution problems.

TABLE I
AGENT ATTRIBUTION PATTERNS REVEALING RELIABILITY CONCERNS

Agent	Users	PRs/User
OpenAI Codex	61,653	13.2
GitHub Copilot	379	133.1
Claude Code	1,643	3.1
Cursor	9,658	3.4

Copilot shows 133 PRs per user versus Codex at 13—likely reflecting labeling inconsistencies.

B. Observed Patterns

Note: These patterns likely reflect validity threats rather than genuine differences.

Critical Limitations: Agent labels are unreliable (metadata/self-reports), statistical tests invalid due to clustering, test detection has 8% false positive rate, and severe confounding prevents causal inference. **Per-agent testing rates.** Table I summarizes per-agent totals and test percentages. We observe a broad range (19% to 98.5%) and a large association (Cramér’s V 0.646).

TABLE II
PER-AGENT PR TOTALS AND TEST-RELATED PR PERCENTAGES

Agent	Total PRs	Test %
OpenAI Codex	814,522	98.46
GitHub Copilot	50,447	79.77
Claude Code	5,137	77.36
Cursor	32,941	18.96
Overall	932,791	93.52

Acceptance. Acceptance (merged/closed) varies by agent. We avoid causal interpretations because usage contexts differ (e.g., prototyping vs. production). We find no monotonic relationship between testing percentage and acceptance percentage, suggesting multiple interacting factors.

Robustness. Excluding the single Devin user (n=903,047) increases Cramér’s V to 0.669. Treating all Codex-labeled

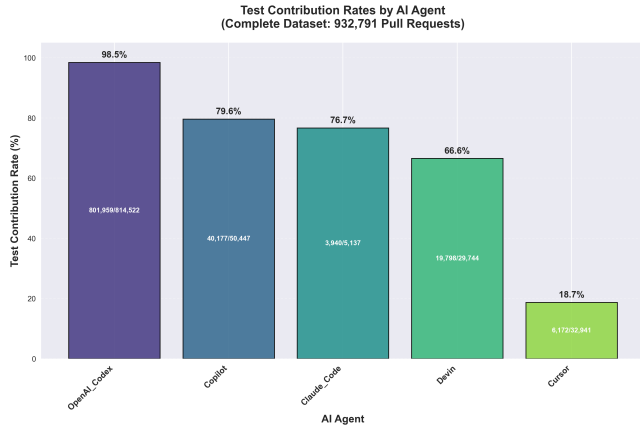


Fig. 1. Test-related PR rates by agent.

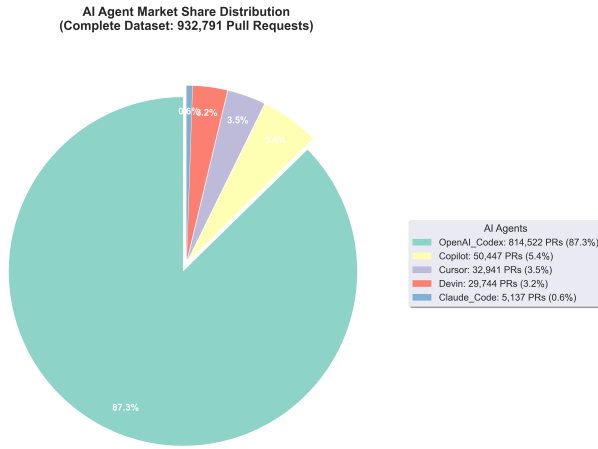


Fig. 2. Agent distribution (counts and percentages).

PRs as Copilot yields a Copilot rate of 97.37% and leaves the overall rate unchanged (93.52%). Excluding Codex (to examine the smaller-agent subset) preserves relative ordering and yields 59.5% overall for the remaining agents.

Description–title matching. On a 3k sample, overall matching rate is 70.07%. By agent: Copilot 88.31%, Devin 90.0% (single user caveat), Claude 77%, Codex 71–74%, Cursor 51.30%. These associations may reflect different usage contexts rather than inherent agent characteristics.

IV. ACCEPTANCE PATTERNS

Acceptance (merged/closed) varies across agents but does not exhibit a simple relationship with testing percentages. Multiple mediating factors (repository policies, task type, developer experience) likely contribute. Observational data cannot disentangle these influences without additional controls. We therefore report acceptance descriptively and do not infer mechanisms.

V. RELATED WORK

Recent code-generation studies emphasize functional correctness and security [2], while empirical work reports developer experiences with assistants such as Copilot [3]. Classic testing literature cautions that coverage does not imply effectiveness [4]. Broader empirical software engineering has explored ownership and review dynamics [5], [7], and change classification [6]. Our population-level analysis complements these by focusing on observed testing presence across agents in the wild.

VI. IMPLICATIONS

For Researchers: Prioritize controlled experiments for behavioral inference. When using observational data, implement comprehensive validity checks and avoid causal claims.

For Practitioners: Interpret observational studies as suggestive rather than definitive. Focus on controlled evaluations for important decisions.

VII. THREATS AND ETHICS

Attribution noise. Agent labels are derived from metadata/self-report. Misclassification likely attenuates differences rather than fabricating 80-point gaps.

Selection effects. Developers and organizations may choose agents by task and context; associations may reflect usage patterns, not capabilities.

Construct limits. We measure test presence, not correctness or coverage. Merge status is a coarse outcome proxy. Repository, language, and team factors are unobserved and may confound associations.

VIII. METHODOLOGICAL REQUIREMENTS

Valid behavioral inference requires:

Attribution Verification: Independent validation achieving ≥90% accuracy with balanced errors across groups.

Clustering Control: Multilevel modeling or cluster-robust methods accounting for user, repository, and organizational structure.

Measurement Validation: Manual validation (n500) across languages and contexts, with bias quantification.

Confounding Assessment: Collection of developer, project, and temporal covariates with appropriate analytical controls.

IX. LIMITATIONS

Validity Threats: Agent labels are unreliable, statistical tests are invalid due to clustering, and severe confounding prevents causal inference. This analysis cannot distinguish agent effects from artifacts.

Ethics. We analyze public data; results are aggregate-only. We avoid judging individuals or vendors and emphasize correlational rather than causal interpretation.

X. REPRODUCIBILITY AND AVAILABILITY

We provide scripts to recompute key statistics directly from the raw CSV (chunked processing), plus a sensitivity script for Codex labeling. Figures are generated from the same pipeline. Data source: AIDev (HuggingFace, config: all_pull_request). Repository link: <https://github.com/ghost1100/MSR2026>.

XI. CONCLUSION

This study demonstrates validity threats in observational AI tool datasets that prevent reliable behavioral inference. Through analysis of AIDev, we show how attribution uncertainty, clustering violations, measurement bias, and confounding create misleading patterns. Our methodological framework establishes requirements for valid inference. This critique targets methodology, not researchers or datasets, aiming to improve research rigor in AI tool evaluation.

ACKNOWLEDGMENTS

We thank H. Li et al. for releasing AIDev and colleagues for feedback. Any errors are our own.

REFERENCES

REFERENCES

- [1] H. Li et al., “The Rise of AI Teammates in Software Engineering (SE 3.0),” arXiv:2409.18925, 2024.
- [2] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021.
- [3] A. Bareiß et al., “Code generation tools in practice: A user study with GitHub Copilot,” in Proc. MSR, pp. 285-297, 2023.
- [4] L. Inozemtseva and R. Holmes, “Coverage is not strongly correlated with test suite effectiveness,” in Proc. ICSE, pp. 435-445, 2014.
- [5] C. Bird et al., “Don’t touch my code! Examining the effects of ownership on software quality,” FSE 2011.
- [6] S. Kim et al., “Classifying software changes: Clean or buggy?,” TSE 2008.
- [7] A. Bacchelli and C. Bird, “Expectations, outcomes, and challenges of modern code review,” ICSE 2013.